中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术
实验项目编号 4 实验名称 流水线处理器设计

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

• **设计**步骤为在代码编写前的准备阶段, 主要包括流水级划分、级间寄存器设计、数据冒险和控制冒险处理方案设计。

• **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog 代码编写, 在原有 CPU 基础上实现流水线化和性能优化。

• **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及根据波形检查停顿、前递和跳转重定向是否正确。

一、 设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 LaTeX 中)

1 流水线 CPU 总体设计

本次实验的主要目标是在前面多周期 CPU 的基础上进一步提升处理器吞吐率。多周期 CPU 虽然已经把一条指令拆分为多个阶段执行, 但同一时刻仍然只有一条指令占用主要数据通路。本次我采用流水线思想, 将取指、译码、执行、访存和写回这几个阶段拆成可以并行推进的流水级, 使不同指令能够在不同阶段同时工作, 从而提升单位时间内完成的指令数量。

在本次设计中, CPU 主要被划分为 IF、ID、EX、MEM、WB 五级。IF 级负责向指令存储器发出取指请求并接收指令; ID 级负责译码、寄存器读数和冒险判断; EX 级负责 ALU 运算、移位、分支判断和跳转目标生成; MEM 级负责访存请求、数据接收以及 load/store 数据处理; WB 级负责最终写回寄存器堆和输出 inst_retire 信息。为了保证各级之间的数据能够稳定传递, 我在相邻流水级之间加入了对应的流水寄存器, 例如 'id_pc'、'ex_inst'、'mem_result'、'wb_rf_waddr' 等信号。

2 数据冒险处理设计

流水线 CPU 中最核心的问题之一是数据冒险。如果后一条指令需要读取前一条指令尚未写回的寄存器, 就会读到旧值。为了减少不必要的停顿, 我在 EX 级加入了前递逻辑。当 MEM 级或 WB 级即将写回的目的寄存器与 EX 级当前指令的源寄存器相同, 并且目的寄存器不是 0 号寄存器时, 就将后级结果直接前递到 EX 级的 ALU 输入。

不过，load-use 冒险无法完全依靠前递解决。因为 load 指令的数据需要等 MEM 阶段从内存返回后才真正有效，如果下一条指令紧跟着使用该结果，在 EX 阶段时数据还没有准备好。因此我在 ID 级加入了 load-use 检测：当 EX 或 MEM 中存在尚未完成的 load 指令，且其写回寄存器被 ID 级指令使用时，ID 级不允许继续推进，从而插入等待周期。这样可以保证指令不会在源数据尚未准备好时进入执行级。

```
wire id_wait_ex_load =
    ex_valid && ex_memread && ex_rf_wen && (ex_rf_waddr != 5'b0) &&
    ((id_rs_used && id_rs == ex_rf_waddr) ||
     (id_rt_used && id_rt == ex_rf_waddr));

wire id_wait_mem_load =
    mem_valid && mem_memread && mem_rf_wen && (mem_rf_waddr != 5'b0) &&
    ((id_rs_used && id_rs == mem_rf_waddr) ||
     (id_rt_used && id_rt == mem_rf_waddr));

assign id_readygo = !(id_wait_ex_load || id_wait_mem_load);
```

图 1: 等待部分代码截图

在判断源寄存器是否被使用时，我没有简单地认为所有指令都使用 rs 和 rt，而是根据指令类型分别计算 ‘id_rs_used‘ 和 ‘id_rt_used‘。例如立即数运算一般使用 rs，不使用 rt 作为源操作数；store 指令需要使用 rt 作为写入数据；部分 load 指令如 lwl/lwr 也会使用 rt 原值参与拼接。这样可以避免对无关寄存器产生错误停顿。

3 控制冒险处理设计

分支和跳转指令会改变 PC，是流水线中的另一个难点。本设计中，分支判断和跳转目标计算主要在 EX 级完成。当 EX 级发现当前指令需要改变控制流时，会产生 ‘ex_redirect‘ 信号，并给出 ‘ex_redirect_pc‘。IF 级检测到重定向后会清空当前尚未确认的取指结果，并将 PC 修改为新的目标地址。

为了避免错误路径上的指令继续向后执行，我还在 IF/ID 寄存器更新时加入了清空逻辑。当发生重定向并且 ID 级当前保存的指令不是重定向目标时，ID 级会被清空。这样可以防止已经取到但不应执行的指令污染后续流水级。

此外，由于当前设计中的取指接口需要等待返回，IF 级自身也被细分成 IF_REQ、IF_WAIT、IF_DONE、IF_BRANCH_WAIT 和 IF_MEM_WAIT 等状态。对于控制流指令和访存指令，IF 级会适当等待后续阶段处理完成，再继续发起下一次取指。这种设计在一定程度上降低了乱序请求带来的复杂度，也让流水线更容易与阻塞式访存接口配合。

二、 开发成果（对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明）

1 流水线寄存器与握手控制

代码中首先定义了各级 ‘valid‘ 信号和对应的状态寄存器。ID、EX、WB 等级使用较简单的 EMPTY/DONE 状态，MEM 级由于需要访存，所以额外包含 MEM_REQ、MEM_WAIT 和 MEM_DONE 等状态。每一级的 ‘allowin‘ 信号由自身状态决定，只有处于空状态时才允许上一级写入新的指令。

级间传递时，我将指令相关信息集中保存在流水寄存器中。例如 ID/EX 寄存器不仅保存 PC 和指令本身，还保存译码结果、寄存器读数、立即数扩展、控制信号和写回目的寄存器。这样 EX 级不再依赖 ID 级的组合译码输出，而是使用已经锁存好的指令上下文。这个设计让每一级的职责更明确，也便于在波形中观察单条指令如何从前级流向后级。

```
reg [31:0] if_pc;

reg [31:0] id_pc;
reg [31:0] id_inst;

reg [31:0] ex_pc;
reg [31:0] ex_inst;
reg [31:0] ex_rdata1;
reg [31:0] ex_rdata2;
reg [31:0] ex_extend;
reg [31:0] ex_lui_extend;
reg [31:0] ex_offset_two_extend;
reg [4:0]  ex_rs;
reg [4:0]  ex_rt;
reg [4:0]  ex_rd;
reg [4:0]  ex_shamt;
reg [5:0]  ex_opcode;
reg [5:0]  ex_func;
reg [4:0]  ex_reg;
reg [25:0] ex_instr_index;
reg [3:0]  ex_mode;
reg [2:0]  ex_alu_op;
reg        ex_alusrc;
reg        ex_branch;
reg        ex_memread;
reg        ex_memwrite;
reg        ex_rf_wen;
reg [4:0]  ex_rf_waddr;
```

图 2: 相关寄存器定义

2 译码与执行级开发

译码部分基本沿用了前几次实验中的指令分类方式，仍然通过 opcode、func 等字段生成 ‘id_mode‘、‘id_alu_op‘、‘id_alusrc‘、‘id_branch‘、‘id_memread‘、‘id_memwrite‘ 和 ‘id_rf_wen‘ 等控制信号。为了适配流水线，我将这些控制信号在 ID/EX 寄存器中保存，并在后续阶段继续使用。

执行级中，ALU 输入首先经过前递选择。对于源操作数 A，如果 MEM 级有非 load 指令即将写回且目的寄存器匹配，则优先使用 MEM 级结果；否则检查 WB 级结果；都不满足时才使用 EX 寄存器中保存的原始读数。源操作数 B 的前递方式类似，只是在立即数指令中会选择扩展后的立即数。这样的优先级可以保证越新的计算结果越先被使用。

```
wire [31:0] ex_forward_rdata1 =
    (mem_valid && mem_rf_wen && !mem_memread &&
     mem_rf_waddr != 5'b0 && mem_rf_waddr == ex_rs) ? mem_result :
    (wb_valid && wb_rf_wen && wb_rf_waddr != 5'b0 && wb_rf_waddr == ex_rs) ? wb_result :
                                                                             ex_rdata1;

wire [31:0] ex_forward_rdata2 =
    (mem_valid && mem_rf_wen && !mem_memread &&
     mem_rf_waddr != 5'b0 && mem_rf_waddr == ex_rt) ? mem_result :
    (wb_valid && wb_rf_wen &&wb_rf_waddr != 5'b0 && wb_rf_waddr == ex_rt) ? wb_result :
                                                                            ex_rdata2;
```

图 3: 前递部分代码截图

分支判断也在 EX 级完成。beq/bne 依赖 ALU 的 Zero 信号，blez/bgtz 等指令则结合 ALU 结果的符号位和 Zero 信号进行判断。REGIMM 类分支使用 slt 风格的结果区分 bltz 和 bgez。跳转目标则根据指令类型在寄存器跳转地址和 J 型拼接地址之间选择。

3 访存级与写回级开发

MEM 级主要负责 load/store 指令与内存总线之间的交互。对于非访存指令，MEM 级很快进入 MEM_DONE，把 EX 级传来的结果继续送往 WB；对于 load/store 指令，则先发起内存请求，等待握手成功后再根据请求类型

进入等待数据或完成状态。load 返回数据时，需要根据 opcode 和地址低两位完成 lb/lbu/lh/lhu/lw/lwl/lwr 等不同形式的数据拼接和符号扩展。

store 写入则仍然沿用前面单周期和多周期 CPU 中的字节写掩码思路，根据 sw、sh、sb、swl、swr 等指令生成不同的 'Write_strb' 和 'Write_data'。这些信号只有在 MEM_REQ 且当前确实为 store 指令时才有效，避免非访存阶段对外部总线产生干扰。

WB 级负责最终写回寄存器堆和生成 inst_retire。由于流水线中同一时刻可能有多条指令存在，inst_retire 必须对应真正到达 WB 级的那条指令。因此我使用 'wb_pc'、'wb_rf_wen'、'wb_rf_waddr' 和 'wb_result' 组合成提交信息。性能计数器中，周期计数每周期加一，提交指令数则在 WB 有效时加一，用来反映流水线实际完成指令的数量。

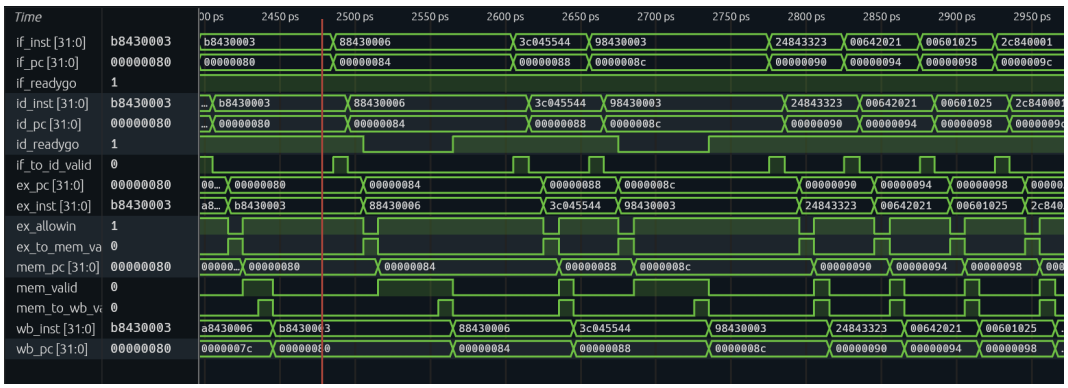


图 4: 多级并行推进的波形图

三、 学习总结（总结设计、开发与调试过程中遇到的问题，及实验中的收获与不足）

1 问题总结（实验过程中遇到的问题、对问题的思考过程及解决方法）

表 1: 实验问题总结

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑电路设计	前递逻辑	运算指令读取到旧寄存器值	流水线中前一条指令尚未写回，后一条指令已经进入 EX	在 EX 级加入 MEM/WB 到 ALU 输入的前递选择
2	行为仿真	load-use 检测	load 后紧跟使用结果时计算错误	load 数据在 MEM 阶段后才有效，无法在下一拍直接前递	在 ID 级检测 EX/MEM 中的 load 相关性，必要时阻塞 ID 级
3	逻辑电路设计	控制流重定向	分支跳转后错误路径指令继续执行	PC 更新后没有同步清空已经取到的错误指令	使用 'ex_redirect' 清空 IF/ID 中不应继续执行的指令

- **流水线调试问题补充。**

流水线调试比多周期 CPU 更容易出现"看起来每个模块都对，但组合起来不对"的情况。因为错误往往不是某一条指令本身的译码问题，而是两条或多条指令之间的时间关系问题。例如寄存器堆读出的值在单独看时没有

问题,但放到流水线中可能已经落后于最新结果。因此调试时我主要观察 PC、各级 valid、各级指令、写回寄存器和前递选择信号,沿着一条具体指令追踪它在各级之间的移动过程。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

通过本次流水线 CPU 实验,我对处理器性能优化有了更直接的理解。多周期 CPU 的优势在于降低单周期路径长度,而流水线 CPU 的优势在于让不同指令重叠执行。这个过程并不是简单地把寄存器插到数据通路中间就结束了,还必须同时处理数据冒险、控制冒险和外部总线等待。尤其是 ready/valid 机制与流水线级间寄存器结合后,控制逻辑明显比前几次实验更复杂。

本次实验的亮点是能够用前递减少大部分数据相关带来的停顿,并且通过 load-use 检测保证结果正确;不足之处在于流水线仍然比较保守,遇到控制流和访存时会进行较多等待,没有实现更积极的分支预测或非阻塞访存。后续如果继续优化,可以从减少控制流等待和改进 IF 级取指策略入手。

表 2: 收获总结

类别	名称	体会	后续计划
设计方法	五级流水线划分	掌握了用级间寄存器保存指令上下文的方法,也理解了流水线吞吐率提升的来源。	后续尝试减少 IF 级等待, 让取指更加连续。
RTL 编写	冒险处理	前递和暂停逻辑需要严格区分不同指令使用的源寄存器, 否则容易产生过度停顿或错误推进。	继续整理指令使用 rs/rt 的规则, 减少冒险检测中的冗余判断。
验证方法	指令级追踪	通过观察各级 PC 和指令, 可以定位流水线中的时序错位问题。	补充连续相关运算、load-use、分支跳转等专项测试。

四、 实验时长

在课后,你花费了大约 70 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	10	主要用于划分流水级和设计冒险处理方案
开发	20	完成级间寄存器、前递、暂停和访存控制
调试	40	通过波形检查数据相关和控制流重定向

表 3: 实验时长